Precision-Weighted Training for Language Models: When Loss and Quality Diverge

Author: Troy Corbin
Research assistance: Claude (Anthropic). Claude contributed to literature connection (linking the mechanism to Predictive Coding), experimental design discussions, data analysis scripting, and drafting this manuscript. All research direction, experiments, and final decisions were made by the author.
Version: 1.0 (working paper)
Date: 2026-04-27
Codebase: Method implementation and evaluation code available at github.com/troycorbinz/precision-weighted-training. The full CLLM v1.5 research repository is private; the public companion repo contains the gain function, layer-gain scaler, A/B evaluation webapp, and configuration needed to reproduce the method.

Abstract

Two language models trained on identical data in identical order can converge to the same smoothed validation loss yet differ noticeably in the quality of what they generate, and the metric most LLM training literature reports won't see it. In this paper *quality* is defined operationally as blind A/B preference rate by a 42-judge panel of humans and foundation-model judges. In a controlled 1.2B-parameter comparison at 3.9B tokens (16.4% of Chinchilla-optimal), a Predictive-Coding-inspired reshaping of the per-token learning signal achieves smoothed val loss within 0.004 of baseline yet is preferred in **59.9% of 784 decisive blind A/B comparisons** across 1,181 judgments by that panel ($p = 2.80 \times 10^{-8}$, two-sided binomial; direction survives every sensitivity filter we apply).

The intervention has two composable parts: a **per-token precision-weighted gain** that redistributes gradient toward surprising tokens (mean-normalized at construction), and a **per-layer divergence-scaled gradient** that amplifies learning where representations are actively revising. Both are cheap (no measured throughput impact) and optimizer-agnostic. Three findings carry through three experimental phases from 50M to 1.2B parameters: (1) **mean-normalization is the critical property**: gain functions that shift the mean loss weight away from 1.0 degenerate training; (2) **loss and output quality diverge at scale**, which is the headline above; (3) **functional layer specialization emerges** under divergence-scaled gradients, with late-block representation divergence growing 387% across training while mid-block stays stable.

The methodological takeaway is as load-bearing as the mechanism: at the parameter and token scales we tested, **aggregate val loss is not sufficient to distinguish a real improvement from no improvement at all**. A 42-judge blind panel of 29 human volunteers (including the

author) and 13 foundation-model judges spanning eleven vendors, recruited both in person and through open online posts, agreed on a quality difference that aggregate loss could not see.

---

1. Introduction

The core of gradient-based language model training has not changed since the original Transformer: compute cross-entropy loss averaged over all valid tokens in a batch, backpropagate, update weights. Every token contributes equally to the gradient; every layer receives gradient proportional to its local error signal and nothing more. This uniform treatment has been the universal default for at least a decade.

We were skeptical of this uniformity for two reasons. First, it is at odds with how biological learning systems adapt: the brain uses dopamine-mediated precision weighting to dynamically modulate how strongly prediction errors drive synaptic change. Surprising signals are amplified, confident predictions are attenuated, and noisy periods are trusted less (Rao & Ballard, 1999; Friston, 2005). Second, the loss signal in language modeling is genuinely heterogeneous: predicting "the" in "the dog sat" (a near-certain bet from frequency alone) is not the same kind of learning as predicting "embryology" in "the subject of developmental embryology" (where many words could fit and the model genuinely has to know the topic). Uniform weighting implicitly claims they are.

This paper develops and empirically characterizes two training-time interventions, per-token precision-weighted gain and per-layer divergence-scaled gradients, that together give the model's own prediction error and representation-change signal the job of shaping the learning signal. Throughout, *quality* refers to the operational measure used in our evaluation: the rate at which one model's outputs are preferred over another's in blind A/B comparisons by a 42-judge panel of humans and foundation-model judges (full setup in Section 7.1). We make three contributions:

1. **A Predictive-Coding-motivated mechanism**: the per-token gain function `gain = 1 + s · precision · centered_error` (with strength scalar `s`, default 1.0) implements the PC precision-weighting principle at the batch-of-tokens level. It is mean-normalized by construction, self-regulating under noisy batches, and composes with any optimizer. The relationship to full PC, and the ways our implementation is a simplification, is made explicit in Section 3.1.
2. **A complementary layer-level mechanism**: per-block forward-pass representation divergence is used to scale parameter gradients after backward. Layers that are actively revising their representations receive amplified gradients; stable layers are attenuated. Like token gain, layer gain's per-block scales are mean-normalized at construction (mean = 1.0 across blocks before clamping) so the mechanism is designed to redistribute learning pressure across the depth of the network rather than to globally rescale gradients. Layer gain is not "precision weighting" in the PC sense (there are no layer-wise prediction errors being precision-modulated); it is a divergence-proportional gradient redistribution inspired by the same "scale learning by where the model is currently revising" intuition (see Section 3.1).

3. **Empirical evidence that the aggregate loss metric is insufficient.** In a controlled 1.2B-parameter comparison trained on identical data in identical order for 30,000 steps (3.9B tokens), baseline and gain-trained models achieve nearly identical smoothed val loss, yet the gain model is preferred 59.9% of the time in decisive blind pairwise comparison across 1,181 judgments by a panel of 29 human and 13 foundation-model judges. Humans and foundation models agree on direction (60.5% vs 59.0% gain preference of decisive judgments) and the result survives every sensitivity filter we apply (excluding speed-clickers, tie-biased judges, partial completions, and all of the above simultaneously). These are large, robust signals that aggregate loss simply cannot see.

Scope and key limitations (detailed in Section 9). The Phase 3 result is a single-seed, single-pair comparison at 16.4% of Chinchilla-optimal training. We do not ablate token gain and layer gain separately at 1.2B scale; both are combined in the gain run. The baseline did not log layer divergences, so we cannot confirm whether the observed layer specialization is caused by layer-gain scaling or would emerge under uniform training as well. The A/B evaluation uses short-form prompts only (the models are too undertrained for long-form), and the foundation-model judges may share biases from overlapping training data. These are important caveats for interpreting the strength of the claims. A Chinchilla-scale follow-up at 1.5B parameters is in progress; an accidental paired ablation within that follow-up (Section 6.4) provides early cross-scale evidence that the layer-gain mechanism's effect depends on layer 0's participation in divergence normalization, but a full 1.5B baseline-vs-gain A/B has not yet been collected.

Methodological positioning. The approach taken here (proposing a mechanism, measuring its signature in training dynamics, making falsifiable predictions, and revising the theory when prediction and observation disagree, as Section 4.2's layer-0 story does) fits within the emerging "learning mechanics" framing of deep learning theory (Simon et al., 2026), which argues that a scientific theory of the field is consolidating around training dynamics, coarse aggregate statistics, and falsifiable quantitative predictions.

The paper is structured as follows. Section 2 positions the work against neighboring approaches in token reweighting, layer-wise gradient modification, and loss-quality decoupling. Section 3 frames the work in the Predictive Coding literature. Section 4 describes both mechanisms and the config surface. Section 5 establishes the mechanism's shape and stability properties at 50M parameters (the runs tagged Phase 1 and Phase 2 in our data files). Section 6 reports the 1.2B comparison (Phase 3) and a forced revision of the layer-gain story by an accidental 1.5B ablation. Section 7 reports the blind A/B preference evaluation in three passes: the headline result and its sensitivity to filters; who the judges were and where they agreed; and where the preference comes from prompt by prompt. Section 8 analyses loss-quality divergence, the episodic-to-semantic trade-off, and training-practicality considerations. Section 9 discusses limitations in full. Section 10 concludes.

2. Related Work

This work sits at the intersection of three threads.

Per-token loss reweighting. The most-cited example is focal loss (Lin et al., 2017), which down-weights confidently classified examples to focus learning on hard ones; it was developed for dense object detection and has since been adapted variously to language modeling. A more recent and directly comparable line modifies the per-token cross-entropy in language-model training itself: Helm et al. (2025) propose Token Weighting for Long-Range Language Modeling, where per-token weights are derived from comparing a long-context model's confidence to a short-context model's confidence on the same token, isolating the tokens that genuinely require long-range information. A separate strand (importance sampling and hard-example mining) adjusts *which* examples are seen rather than *how strongly* each contributes. Our token-level mechanism belongs to this family but differs in two specific ways. It is **mean-normalized by construction** (the average gain across tokens equals 1.0 before clamping), so the mean loss magnitude is preserved rather than reduced; and its weight is derived from **batch-level variance** of the per-token loss in a single forward pass (no second model, no external signal), so a tightly clustered batch concentrates updates while a noisy batch applies them more uniformly. Section 5.1 shows experimentally that mean-normalization is the property separating a stable variant (linear, sigmoid, our precision-weighted variant) from a destabilizing one (focal at 50M parameters degenerates output without it).

Layer-wise gradient modification. Layer-wise learning-rate decay, GradNorm (Chen et al., 2018), and large-batch optimizers like LARS / LAMB (You et al., 2017, 2020) all modify how gradient flows through depth, but they do so via hand-designed schedules or per-parameter statistics rather than a per-step measurement of how much each block is currently revising its representation. We are not aware of prior work that uses forward-pass representation divergence `||x_out - x_in|| / ||x_in||` as a per-block, per-step gradient scale.

Loss-quality decoupling. That preference quality and perplexity can move differently is established in the RLHF literature: preference-tuned models often gain in human judgment while perplexity is flat or worsens (Ouyang et al., 2022; Bai et al., 2022). What is new in this paper is that we observe the same decoupling **during pretraining**, on identical data and identical seeds, between two runs that differ only in the per-token gain function: no preference data, no RL, no instruction tuning. A 0.004-nat aggregate gap accompanying a 59.9% blind preference is, as far as we have found, the strongest reported instance of this decoupling in a controlled same-data pretraining comparison.

Positioning. Compared to focal loss, our gain *preserves* rather than reduces average loss. Compared to RLHF, it sits inside the standard pretraining loop with no preference data. Compared to layer-wise learning-rate schedules, it scales gradient by a measured per-step signal rather than a hand-tuned schedule. The contribution is not a new optimizer or architecture; it is a small, mean-preserving, variance-derived modification to the cross-entropy reduction step, plus a divergence-derived per-block gradient scale, that together produce a measurable preference shift at a matched aggregate-loss budget.

3. Background: Predictive Coding and Precision Weighting

Predictive Coding (PC) is a framework in computational neuroscience that models perception and learning as hierarchical prediction error minimization (Rao & Ballard, 1999; Friston, 2005; Millidge et al., 2021). At every level of the hierarchy, a neural population predicts the activity of the level below; the mismatch between prediction and observation is the prediction error, which drives both inference (updating beliefs to explain the input) and learning (updating the weights that generate predictions).

A central concept in PC is **precision weighting**. The prediction error signal is not treated uniformly; it is multiplied by a precision term, which in probabilistic formulations is the inverse variance of the error. Precision represents the reliability of the signal: when the system believes the error is informative (low variance, consistent signal), precision is high and the error drives learning strongly. When the system believes the error is unreliable (high variance, noisy signal), precision is low and the error is discounted. Precision is hypothesized to be mediated neurally by neuromodulators including dopamine, linking it directly to attention, salience, and the modulation of synaptic plasticity (Friston, 2009).

This framework maps onto language model training cleanly at the level of the core principle. The per-token cross-entropy loss *is* a prediction error. A precision-weighted gradient update on that error is therefore a natural translation of PC's core idea into the LM setting, albeit a structurally simpler one than full PC (Section 3.1 details the differences). The question is what "precision" means in a training batch. We use the simplest possible definition: `precision = 1 / var(per_token_loss)`. A batch with tightly clustered per-token losses has high precision (consistent signal), a batch with highly variable losses has low precision (noisy signal), and the per-token deviation from the batch mean provides the centered error that precision multiplies. The result is a gain function with two desirable properties simultaneously: (1) it redistributes gradient toward surprising tokens, and (2) it does so more strongly when the batch's error signal is internally consistent, and less strongly when it is not.

A related but distinct intuition motivates our layer-level mechanism. During the forward pass, each transformer block transforms an input representation `x_in` into an output representation `x_out`. The relative change `||x_out - x_in|| / ||x_in||`, which we call *divergence*, is a measure of how much that block is actively revising the representation at this step. A high-divergence block is a layer currently doing substantial work; a low-divergence block is a layer that has stabilized. Amplifying gradient on high-divergence layers and attenuating it on low-divergence ones directs learning to where representational change is already happening. This is not precision weighting in the PC sense (there is no layer-wise prediction error being precision-modulated); it is a divergence-proportional gradient redistribution, motivated by the same intuition, "scale learning by where the model is already revising", but implemented through a different signal.

The two mechanisms target different axes. Token gain (precision-weighted) reshapes *what* the model learns from, which token errors drive the most update. Layer gain (divergence-proportional) reshapes *where* the learning lands, which blocks get the strongest parameter changes.

They compose straightforwardly because both are loss-level / gradient-level modifications that do not interact with the optimizer or the model architecture.

3.1 Relationship to full Predictive Coding

Our method borrows the core PC principle, scale the learning signal by the reliability of the error, but simplifies the full framework in several ways. Readers familiar with PC should understand these differences before evaluating the mechanism's claims.

We use batch-level precision, not per-unit precision. In full PC, every error signal at every hierarchical level carries its own precision value, giving the framework fine-grained control over which specific errors drive learning. Our gain function computes a single scalar precision `1/var(loss)` per batch and applies it uniformly; per-token variation in the final gain weight comes entirely from the centered error $(\ell_i - \ell)$, not from per-token precision. This is a coarser implementation of the same principle.

Precision is computed, not learned. In full PC, precision is itself a parameter of the generative model, updated over time based on how consistent the error signal proves to be. Our precision is recomputed from batch statistics at every step with no memory, it has no notion of "some tokens are reliably predictable and should have persistently high precision." A fully-learned precision parameter is a natural extension but is not what we have implemented.

Precision weighting is applied at the output only. PC is fundamentally hierarchical: each layer predicts the level below, each layer has its own prediction error, and each layer's error is precision-weighted. Our token-level gain operates at the cross-entropy loss (the output). The layer-gain mechanism (Section 4.2) is a complementary tool that acts on gradient magnitudes per block, but it does not use layer-wise prediction errors in the PC sense, it uses representation *change* as a proxy for where the model is actively revising.

Mean-normalization is an engineering decision, not a PC property. PC simply multiplies error by precision and allows total magnitude to fluctuate. We center our gain at 1.0 so the mean loss weight is preserved across the batch, a property chosen to keep LM training stable (see Section 5.1 for experiments showing what happens when this property is violated). This mean-normalization is the difference between our precision-weighted variant and the failed focal and sigmoid variants that do not enforce it; it is our addition, not PC's.

No neurobiological grounding. PC's precision is hypothesized to be implemented by specific neuromodulator systems (notably dopamine) gating plasticity at specific synapses. Our gain function has no such mapping; it is a statistical operation on batch losses. The theoretical inheritance is conceptual, not mechanistic.

Despite these simplifications, the mechanism faithfully implements what we consider the core PC intuition: a multiplicative, variance-sensitive modulation of the learning signal that amplifies consistent evidence and discounts noisy evidence. The empirical finding that *mean-normalization is the critical property* (Section 5.1) is itself a useful refinement to the PC-inspired recipe when applied at the batch level in language model training.

4. Method

4.1 Per-token precision-weighted gain

We compute the standard per-token cross-entropy loss with `reduction='none'`, apply a detached gain multiplier, and then take the mean over non-padding tokens:

```
per_token_loss = F.cross_entropy(logits, targets, reduction='none', ignore_index=pad_id)

gain = gain_fn(per_token_loss)   # detached, shape (N,)

loss = (per_token_loss * gain)[non_pad_mask].mean()
```

The precision-weighted gain is:

$$\mathbf{gain}_i = \mathbf{clamp}\left(1 + \mathbf{s} \cdot \frac{\ell_i - \bar{\ell}}{\mathrm{Var}(\ell) + \varepsilon}, c_{\min}, c_{\max}\right)$$

where ℓ_i is the per-token loss for token i, $\bar{\ell}$ is the batch mean, $\mathrm{Var}(\ell)$ is the batch variance, $\mathbf{s}$ is a strength scalar (default 1.0), $\varepsilon = 10^{-6}$ for numerical stability, and $c_{\min} = 0.1$, $c_{\max} = 5.0$ are safety clamps that rarely bind in practice. Implementation: `PrecisionWeightedGain` class in src/gain_functions.py.

Key properties:

- **Mean-normalized by construction (pre-clamp).** Since $\sum_i(\ell_i - \bar{\ell}) = 0$ exactly, the mean of the pre-clamp gain weight across the batch is exactly 1.0. The mechanism is therefore designed to redistribute learning pressure across tokens rather than to globally rescale every token. The safety clamps can introduce small post-clamp deviations from mean = 1.0 when they bind, though they rarely bind in practice (Section 5.1). This pre-clamp mean-normalization at the loss-weight level does not constrain the post-backward gradient norm, which depends on the per-token loss reshaping, the optimizer interaction, and the layer-wise parameter structure (see Section 6.2 for the observed grad-norm behavior).
- **Self-regulating.** In a high-variance (noisy) batch, precision $1/\mathrm{Var}(\ell)$ is low, so the gain is close to uniform. In a low-variance (consistent) batch, precision is high and redistribution is strong. The mechanism trusts consistent signals and discounts noisy ones.
- **Detached.** Gain weights do not contribute their own gradients. The gain function is a non-differentiable reshaping of the loss landscape, not a learned component.
- **Optimizer-agnostic.** The gain modifies loss before backward. Whatever optimizer consumes the resulting gradients is unaffected. We use Muon for 2D weight matrices and AdamW for embeddings and scalar parameters; the gain interacts with both without modification.

4.2 Per-layer divergence gradient scaling

During the forward pass, each block records its representation divergence:

```
# In forward_blocks, outside checkpoint scope, detached:

div = (x_out - x_in).norm() / (x_in.norm() + 1e-8)

self._layer_divergences.append(div.item())
```

After `loss.backward()` and before gradient clipping, the `LayerGainScaler` applies a mean-normalized scale to each block's parameter gradients. Let d_i be the divergence of block i and $\bar{d}$ the mean divergence across all blocks (including layer 0, see the discussion below). The scale is:

$$\text{scale}_i = \text{clamp}\left(1 + \mathbf{t} \cdot \frac{d_i - \bar{d}}{\bar{d}},\ m_{\text{min}},\ m_{\text{max}}\right)$$

with strength $\mathbf{t} = 0.5$, $m_{\text{min}} = 0.1$, $m_{\text{max}} = 3.0$ (default). Each block's parameters are multiplied by its scale in-place:

```
for name, param in model.named_parameters():

    if param.grad is None: continue

    for prefix, s in scale_map.items():

        if name.startswith(prefix):

            param.grad.mul_(s)

            break
```

Implementation: `LayerGainScaler` class in src/layer_gain.py.

Scope of the per-block scale. The prefix match applies scale_i uniformly to every parameter inside block i, pre-norm, attention projections (Q, K, V, O, including QK-norms), post-norm, and the FFN / MoE. Attention and FFN within a block therefore learn at a common effective rate. The divergence d_i driving the scale is correspondingly compound: it measures the block's full residual contribution (attention-residual composed with FFN-residual), not the two sub-blocks independently. Precision-weighted gain at the token level also propagates through attention via normal backpropagation, in proportion to each token's participation in the gradient path. Splitting the block divergence into attention-specific and FFN-specific signals, and scaling each sub-block's gradients by its own divergence, is a natural refinement left to future work.

Role of layer 0. Layer 0 is structurally unique: it bridges token embeddings and transformer representations, and its forward-pass divergence is dominated by this embedding-to-representation transformation rather than ongoing refinement. Across the Phase 3 gain run, layer 0's divergence sat several-fold above the mean of the other 19 layers across training (Section 6.3; selected trajectory in Appendix D, full per-layer data in JSON) and consistently saturated the $m_{\text{max}} = 3.0$

clamp. This saturation is not a pathology; it is the signal the mechanism uses. Layer 0's high divergence inflates the normalization factor $\bar{d}$, which compresses the scales for stable mid-layers below 1.0. This implicit attenuation of layers whose representations have already settled is the mechanism's signature behavior. A paired ablation at 1.5B parameters (Section 6.4) confirmed this interpretation: excluding layer 0 from the mean computation, which we initially hypothesized would be a cleaner treatment of its saturation, caused token entropy to collapse once training entered the stable-LR phase. Restoring layer 0 participation restored the mechanism. All layers therefore participate in the mean; the $m_{\max}$ clamp handles layer 0's natural saturation without special casing.

Interaction with adaptive optimizers. *(This paragraph is the technically densest passage of the paper; readers focused on empirical results can skip ahead to Section 5.)* Both AdamW (acting on embeddings, biases, and norms) and Muon (acting on 2D weight matrices via Newton-Schulz orthogonalization) are approximately scale-invariant per parameter: multiplying a parameter's gradient by a constant c propagates into the optimizer state in ways that largely cancel out in steady state. The portion of the layer-gain signal that survives this cancellation is the *temporal-derivative-of-divergence* component, the ratio of the current step's scale to its EMA-smoothed recent baseline (with $\beta_2 = 0.999$, that EMA spans roughly 1000 steps). When a layer's divergence is rising or falling rapidly relative to its recent average, the surviving signal pushes its updates above or below their otherwise-normalized magnitude; when divergence is stable, layer-gain contributes little. The Section 6.4 ablation supports this reading directly: a strong-cancellation interpretation cannot explain why layer 0 inclusion vs. exclusion produces qualitatively different training trajectories. The mechanism's contribution is therefore better understood as "amplify layers whose divergence is currently changing fastest" than as "amplify layers whose absolute divergence is highest." Both framings predict that layer-0 inclusion matters; only the temporal-derivative reading is consistent with adaptive optimizers absorbing constant-scale boosts.

4.3 Compositional design

The two mechanisms are deliberately independent and act at different points in the pipeline:

```
tokens → forward pass → logits → per-token CE → gain → loss
                                                        |
                                                    backward
                                                        |
                                            per-block gradients
                                                        |
                                                  layer gain scaling
                                                        |
                                                gradient clipping → optimizer step
```

Token gain happens before backward; layer gain happens after. Neither knows about the other. Both are mean-normalized at the scalar-weight level before clamping, token gain has mean 1.0 across the batch, layer gain has mean 1.0 across blocks, so both are designed to redistribute learning pressure rather than globally amplify or suppress it. The post-backward gradient norm is not held constant by this pre-clamp mean-normalization (Section 6.2). Both mechanisms are null-safe: a run with no gain configured incurs zero runtime cost.

Config surface (under `training` in the model's `config.json`):

```
{
  "training": {
    "gain_function": "precision",      // "none" | "linear" | "focal" | "sigmoid" | "prec
    "gain_config": {"scale": 1.0, "clamp_min": 0.1, "clamp_max": 5.0},
    "layer_gain": {
      "enabled": true,
      "strength": 0.5,
      "min_scale": 0.1, "max_scale": 3.0,
      "exclude_layers": []
    }
  }
}
```

5. Establishing the mechanism at small scale

The first two experimental phases established the *shape* properties of per-token gain at 50M parameters and short training runs. They are summarized here; the full Phase 1/2 results are documented separately in gain-function-experiment-phase1.md. We continue to refer to these runs as "Phase 1" and "Phase 2" because that is how they are tagged in the data files and in the W&B logs cited throughout this paper.

5.1 Mean-normalization is the critical property (Phase 1, 50M params)

Four variants trained on identical data (FineWeb Edu, 2.5GB) with identical hyperparameters; only the gain function differed:

- **A0: Uniform.** Standard cross-entropy, control.
- **A1: Linear normalized.** `gain = loss / mean(loss)`, clamped [0.1, 5.0]. Mean-normalized by construction.
- **A2: Focal loss** (Lin et al., 2017). `gain = (1 - p_correct)^γ`, γ = 2. Suppresses confident predictions. The foundational computer-vision approach to non-uniform loss weighting.
- **A3: Sigmoid.** Smooth S-curve centered on batch mean loss. Bounded range [0.5, 1.5]. Mean-centered but not mean-normalized; the gain/mean value drifts during training.

Final val losses were tight: A0 = 6.152, A1 = 6.137, A2 = 6.181, A3 = 6.148. But output quality at step 5000 differed sharply. A2 (focal) and A3 (sigmoid) produced degenerate output (heavy numeric/tabular patterns; near-pure "the the the" loops at A3's worst point) that only partially recovered by step 5000. A1 (linear normalized) produced qualitatively more diverse text than A0.

The key Phase 1 finding was that **only mean-normalization works**. A2's gain/mean drifted to ~0.92 (systematically suppressive) by step 5000. Focal loss's suppressive mechanism attenuates gradients on confident predictions, which at early training are the foundational tokens ("the", "is", "a"). A3's gain/mean drifted from 1.08 (amplifying) early to 0.90 (suppressive) late; it never stabilized. Any shift of gain/mean away from 1.0 caused degeneration. This motivated the move from linear normalization (A1) to the variance-sensitive precision-weighted formula used in Phase 3, which preserves A1's mean-normalization property while adding self-regulation through the batch variance term.

5.2 Clamp range affects stability, not aggregate loss (Phase 2, 50M params)

Three variants of A1 with different clamp ranges: **B0** [0.1, 5.0] wide, **B1** [0.5, 2.0] conservative, **B2** [0.8, 1.2] tight. Final val losses converged to within 0.005 of each other. Clamp range did not affect aggregate loss.

However, stability diverged. B0's wide clamps allowed sporadic degeneration in later training (gain values occasionally reaching 2.0+ during the decay phase). B2's tight clamps produced repetitive pattern collapse, with so little redistribution that the gain function's benefit vanished. B1's conservative clamps produced the most stable output. Phase 2's conclusion was that **conservative clamps produce stability without sacrificing the diversity advantage**.

5.3 What carries into the 1.2B comparison

- Mean-normalization is the critical property, `mean(gain) = 1.0` must hold.
- Clamp range affects late-training stability, not final loss.
- Loss and output quality can diverge meaningfully even at small scale.
- The mechanism is optimizer-agnostic (Phase 1/2 used the same Muon + AdamW stack as Phase 3).

Precision-weighted gain (the Phase 3 formulation) supersedes A1 because it preserves the useful pre-clamp mean-normalization property A1 also has, while adding **variance-sensitive self-regulation** through the precision term: redistribution is strongest when batch losses are internally consistent (low variance, high precision) and weakest when they are noisy. A1's `loss / mean(loss)` form has no analogous self-regulation; it redistributes by the same shape regardless of how noisy the batch's error signal is.

6. Loss and quality diverge at 1.2B scale (Phase 3)

6.1 Setup

Model. 20-layer transformer, 1024 embedding dim, 16-head grouped-query attention (8 KV heads, GQA 2:1), 4-expert MoE FFN (top-2 routed, DeepSeek-V2 style shared expert), block size 2048, DeepSeek-V2 BPE tokenizer (100,002 base tokens, plus added `<pad>` and `<unk>` for 100,004) with 27 added emotion tokens, for a final vocabulary of 100,031. Approximately **1.2B parameters** total. Architecture unchanged between runs.

Training. 30,000 steps with effective batch size 131K tokens (batch_size 2, grad_accum 32, block size 2048). Total training tokens: **3.93B** (16.4% of Chinchilla-optimal for this parameter count; 20× params ≈ 24B tokens would be Chinchilla-optimal; Hoffmann et al., 2022). Muon optimizer for 2D weight matrices (Jordan et al., 2024), AdamW (lr 3e-4) for embeddings and norms. Warmup-Stable-Decay scheduler with 500-step warmup, no cooldown (`decay_fraction = 0.0`). Label smoothing 0.1. BF16 precision on RTX 5090.

Data. 13-dataset training suite (FineWeb Edu, Wikipedia, Gutenberg, multiple conversation / instruct / code corpora) loaded via a deterministic `SequentialCurriculumSampler` with fixed seed 1337. This is important: **both runs saw identical data in identical order**. Any behavioral difference between the two models is therefore attributable to the training-signal intervention, not to data ordering.

Hardware. Both runs on a single NVIDIA RTX 5090 (32GB VRAM). Training throughput peaked at ~6,400 tokens/sec with sustained end-to-end throughput of ~6,000 tok/s; wall-clock time was approximately 7.5 days per run including periodic evaluation, checkpointing, and occasional restart overhead. (Phase 1 and Phase 2 experiments were conducted on an RTX 3090.)

Conditions. Two runs, identical in every respect except: - **Baseline (cllm-v1.5-025):** `training.gain_function: "none"`, `training.layer_gain.enabled: false`. - **Gain (cllm-v1.5-026):** `training.gain_function: "precision"`, `training.layer_gain.enabled: true`.

6.2 Aggregate loss is indistinguishable

Metric	Baseline (025)	Gain (026)	Δ (Gain – Baseline)
Final train loss (step 30000)	3.717	3.756	+0.039 (baseline lower)
Final val loss (step 30000)	4.082	3.823	−0.259 (gain lower)
Last-10 checkpoints val mean (smoothed)	**3.946**	**3.950**	**+0.004 (negligible)**
Mean grad norm	1.738	2.497	Higher but stable (std 0.210 vs 0.204)
Token entropy	7.214	7.425	+0.211 (more diverse output)

The single-point val loss at step 30,000 flatters the gain run (baseline landed on a noisy spike, the gain run on a low point); the smoothed last-10-checkpoint mean is the correct summary. On that measure, **the two runs are statistically indistinguishable on aggregate val loss** (0.004 difference, well inside step-to-step noise ±0.2). Step-to-step train–val gap measurements are similarly noisy. Individual-step gap ratios between the two runs fluctuate between ~0.2× and ~5× depending on which step is sampled, and rolling averages show the gaps are essentially equivalent. We do not claim a generalization-gap advantage from loss-level metrics at this scale; the separation between the two runs is visible in preference evaluation (Section 7), not in aggregate loss.

Grad norm behavior is notable. The gain run's grad norm mean (2.50) is higher than baseline's (1.74), but its standard deviation is slightly *lower* (0.20 vs 0.21). Pre-clamp mean-normalization of the layer-gain scales (Section 4.2) is at the per-block scalar-weight level and does not by itself fix the post-backward gradient norm; the higher mean here reflects the interaction between divergence-weighted scales, layer-wise parameter structure, and the Muon / AdamW optimizers. The point worth emphasizing is **stable redistribution rather than constant total magnitude**: variance does not inflate, no instability was observed across 30K steps.

6.3 Functional layer specialization emerges

The gain run logged per-layer representation divergence every step. This is not just a training-time diagnostic; it is a direct observation of how the model allocates its representational work across depth, and how that allocation changes over training.

The headline finding is **continuous specialization across the full training run, not convergence to a fixed profile**:

Step	L0	L3	L7	L10	L15	L19	L0 / sample mean
1,000	1.77	0.18	0.14	0.14	0.12	0.12	12.4×
5,300	1.30	0.47	0.18	0.17	0.14	0.20	5.6×
10,000	1.23	0.57	0.18	0.16	0.15	0.36	4.3×
15,600	1.34	0.65	0.17	0.14	0.13	0.50	4.2×
20,000	1.36	0.70	0.18	0.14	0.13	0.54	4.1×
25,900	1.43	0.72	0.18	0.14	0.13	0.59	4.1×
29,900	1.44	0.71	0.18	0.14	0.12	0.60	4.1×

The rightmost column is L0 divided by the mean of the five sample layers shown in the table (L3, L7, L10, L15, L19). The ratio against the full L1–L19 mean follows the same decay trajectory; Appendix D gives an abridged print trajectory and the complete per-layer profile is available in JSON.

Layer 3 grew from divergence 0.184 at step 1K to 0.715 at step 30K, a **288% increase**, showing clear emergent specialization. Layers 1, 2, and 3 all grew substantially over training (L1: +132%, L2: +276%, L3: +288%), indicating the early block group as a whole is where representational revision concentrates.

Layer 19 (the final block) grew from 0.123 to 0.597, a **387% increase**, and was *still growing* at step 30K, though the rate had slowed (0.50 → 0.54 → 0.59 → 0.60 across the final 15K steps). Late-stage representation refinement is active but decelerating.

Middle layers (L7–L18) stayed in a low divergence band (~0.09–0.32) with no sustained growth. These are the "refinement" layers, making consistent small adjustments.

Early-transitional band (L4–L6) sits between the rapidly-growing L1–L3 and the stable middle, final divergence 0.35–0.42, a sub-band within the broader early zone rather than a separate zone.

Layer 0 remained the structural outlier (1.2–1.8 across training, sitting at 1.3–1.5 for most of the run after an early-step peak at 1.77). As the embedding-to-representation bridge, its ratio fell from ~12× the sample mean at step 1K to ~4× by step 10K and then stabilized. Its consistent saturation at $m_{\max} = 3.0$ is the mechanism signal described in Section 4.2, the driver of mid-layer scale compression.

Three functional zones emerged over training: **early** (L0–L6, with L0 a structural outlier at 1.4, the strongly-growing L1–L3 finishing at 0.71–0.94, and an L4–L6 transitional sub-band at 0.35–0.42), **mid** (L7–L18, fluctuating between 0.09 and 0.32 with no sustained growth), and **late** (L19, growing from 0.12 to 0.60 with the rate decelerating but not yet plateaued at 30K). This tri-zone structure is not imposed architecturally; the model has no notion of zones. It emerges as a consequence of the training signal's interaction with the loss landscape. We believe the layer-gain mechanism's directed-gradient property accelerates this emergence, though we do not have a baseline comparison for layer divergences (baseline did not log them; the metric was introduced for the gain run). Percent increases are computed from unrounded W&B values; table values above are rounded to two decimals.

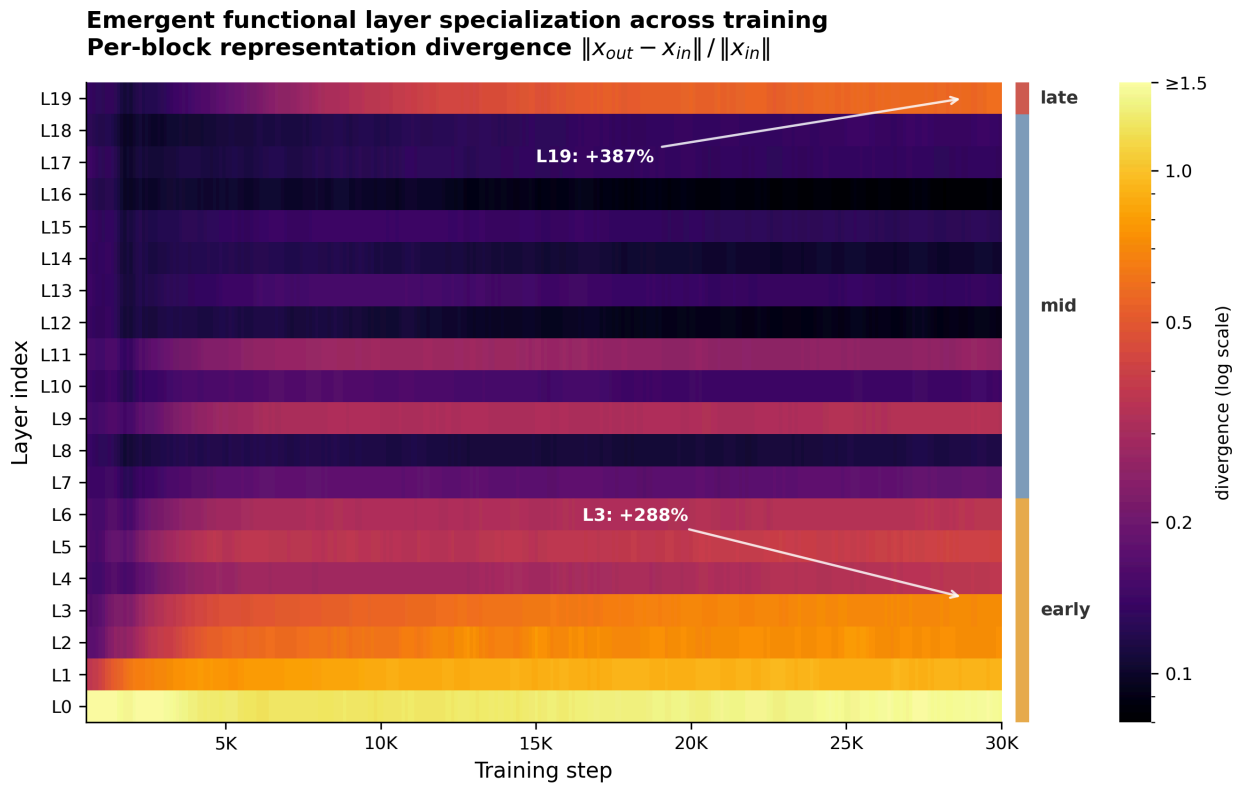

Figure 1. Layer divergence trajectory for the gain run (cllm-v1.5-026). Each cell shows `‖x_out - x_in‖ / ‖x_in‖` for one transformer block at one training step (smoothed with a 5-step window, log-scaled color). The three functional zones are annotated on the right.

6.4 Layer 0's role: a forced revision (1.5B ablation)

A Phase 3 scale-up to 1.5B parameters (24 layers × 1280 embedding dim, same gain configuration, same data and seed as Phase 3) was initiated as a Chinchilla-scale replication. Two consecutive runs, differing only in whether layer 0 participated in divergence-mean normalization, produced a clean accidental ablation of the mechanism.

Setup. Both runs used identical architecture, data, seed, and scheduler. The only difference:

- **cllm-v1.5-028**: `layer_gain.exclude_layers: [0]`, layer 0 removed from the normalization mean and held at scale = 1.0, based on the initial (incorrect) interpretation that layer 0's saturation at $m_{\max}$ was a pathology.
- **cllm-v1.5-029**: `exclude_layers: []`, layer 0 included in the mean, matching the Phase 3 gain run (cllm-v1.5-026).

Warmup was 4000 steps; the stable-LR phase began at step ~5K.

Result at matched step 7000.

Metric	028 (L0 excluded)	029 (L0 included)	026 reference (1.2B)
token_entropy	5.47	**7.04**	7.07
train_loss	4.91	4.96	4.80
val_loss	5.00	5.02	4.78
layer_gain/scale_std	0.22	0.33	0.46

The entropy gap between 028 and 029 at matched step and matched architecture is **+1.57 nats**, a qualitative difference, not a fluctuation. Losses are essentially unchanged between the two (0.05-nat on train, 0.02 on val), so the divergence is entropy-specific rather than a generalized degradation. 028's trajectory has the characteristic shape of a method losing its effect: entropy rose during warmup, peaked at step 5K right after warmup ended, and declined monotonically through step 7K. 029's trajectory stays in the 6.7–7.2 nat band across steps 4K–7K, tracking the shape of the 1.2B reference run 026.

Interpretation. Layer 0's high divergence is the signal the mechanism uses, not noise to be filtered out. With layer 0 in the mean, $\bar{d}$ is inflated and the stable mid-layer scales compress below 1.0, attenuating learning on layers whose representations have already settled. With layer 0 excluded, $\bar{d}$ drops to the mid-layer range and those layers get scales centered on 1.0; the implicit attenuation is lost, and the model defaults to logit-sharpening once training exits warmup. The `scale_std` row of the table tells the same story at the gradient level: without layer 0's participation, the spread of per-layer scales collapses by roughly one-third.

Status and limits. 029 is still training at time of writing. The 1.5B run has not yet reached Chinchilla-optimal scale; we do not claim that the A/B preference advantage of Section 7 replicates at this scale until a completed 1.5B baseline-vs-gain comparison is collected. What the step-7K ablation does show cleanly, at a second scale and second architecture, is that **layer 0's participation in divergence normalization is necessary** for the layer-gain mechanism to function as intended. This result supersedes the "layer 0 excluded" formulation of an earlier draft of this paper and is the basis for the `exclude_layers: []` default in the public code release.

7. Blind A/B Preference Evaluation

A single-scalar loss metric cannot distinguish between qualitatively different generations. For the 1.2B comparison we built a preference-evaluation protocol designed to answer: *do external judges prefer the gain model's outputs over the baseline's, when they do not know which is which?*

7.1 Setup

Question set. 32 prompts across 7 categories: factual (6), reasoning (6), creative (5), conversational (5), structured_output (5), instruction_following (3), world_knowledge (2). Questions were simple and short ("What is the capital of Japan?", "Write a short poem about the ocean.")

because the models are undertrained (16.4% of Chinchilla-optimal) and would not produce coherent long-form outputs. The complete prompt list is in Appendix B.

Generations. For each question, each model produced 3 independent completions at temperature 0.7, top-k 40, max 200 tokens. This gives 32 × 2 × 3 = 192 candidate responses. The webapp draws a unique `(a_gen_idx, b_gen_idx, left_is_a)` pairing per (judge, question) using a deterministic per-judge RNG, so two judges seeing the same question typically see different generations and judges cannot learn a left/right positional heuristic.

Judges. 42 judges total, a mix of people the author knows in person and volunteers recruited via posts on r/MachineLearning and r/SampleSize, plus a panel of foundation-model judges. The final composition:

- **29 human judges** (the author plus 28 volunteers). Self-reported background per the optional demographic survey, which all 29 humans completed: 15 non-technical, 12 technical without ML background, 2 ML-or-CS-research (the author and one self-reported).
- **13 foundation-model judges spanning eleven vendors** (Anthropic, OpenAI, Google, DeepSeek, Meta, xAI, Alibaba (Qwen), Moonshot (Kimi), MiniMax, Mistral, Zhipu), with two judges each from DeepSeek (v3 and v4 Flash) and Meta (Llama 4 Maverick and Muse Spark). Each received the same pairwise comparison interface as humans via a standardized copy-to-clipboard prompt; foundation-model judges did not fill out the demographic survey and appear as "—" in the background column of Appendix C.

Rubric. Each first-time judge saw an instruction modal asking them to score on four axes appropriate to early-training output: relevance, coherence, diversity, and human-likeness. The full verbatim instruction text and the demographic survey are reproduced in Appendix B.1 and B.2; the rubric is what the paper means *operationally* by quality.

Judges were asked to evaluate all 32 pairings; 35 of 42 completed all 32, the remaining 7 partially. Judges could choose "left", "right", or "tie". Model identities were revealed only after submission. The total yielded **1,181 judgments**. Headline numbers below use all judgments; Section 7.2 verifies that the direction is preserved when we exclude partial completions, tie-biased judges, and human speed-clickers.

Interface. A local Flask webapp (eval/ab_compare.py) presented pairings and recorded judgments as JSON. The webapp tracked `left_is_a` per pairing; the post-hoc `winner` field was computed from `choice` and `left_is_a` so that "a" always refers to model A (baseline) and "b" to model B (gain), independent of display position.

7.2 The headline result, and how robust it is

	Baseline (A)	Gain (B)	Tie
Total	314 (26.6%)	**470 (39.8%)**	397 (33.6%)
Of decisive (N = 784)	40.1%	**59.9%**	—

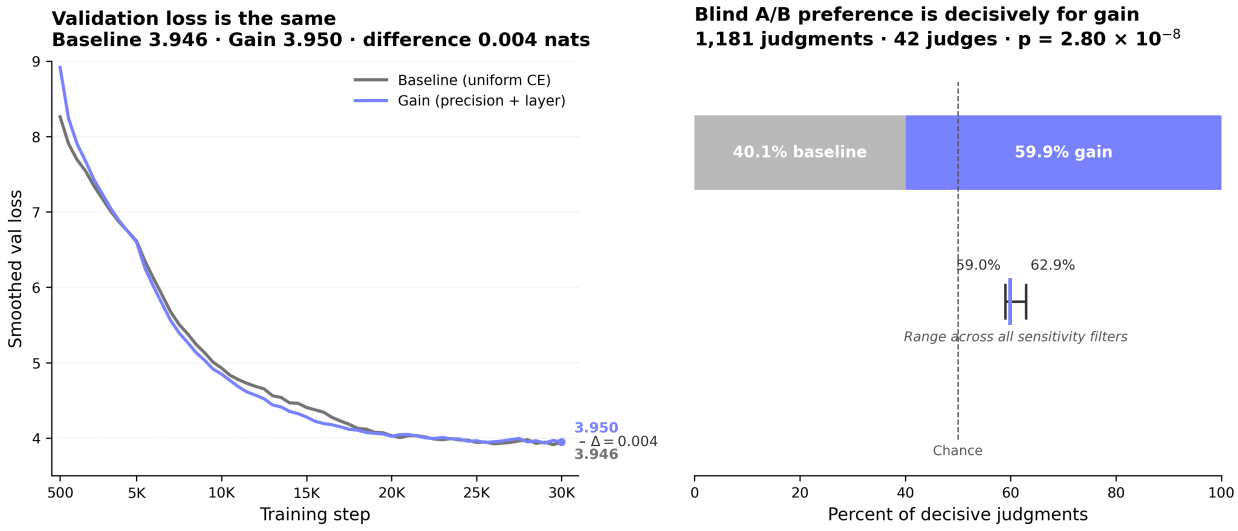

Figure 2. Aggregate validation loss is the same; blind A/B preference is decisively for gain. *Left:* smoothed val-loss trajectories of the baseline (cllm-v1.5-025) and gain (cllm-v1.5-026) runs across 30,000 training steps converge to within 0.004 nats and are visually indistinguishable from ~step 5,000 onward. *Right:* 1,181 blind A/B judgments by a 42-judge panel split 40.1% / 59.9% on the 784-decisive subset ($p = 2.80 \times 10^{-8}$); the whisker spans the B%-decisive range across all sensitivity filters (59.0% FMs only → 62.9% strictest exclusion). Source data: paper/data/phase3_loss_trajectories.json and paper/data/phase3_ab_preference.json.

Two-sided binomial test against H_0: $p = 0.5$ on decisive judgments: $\mathbf{p = 2.80 \times 10^{-8}}$, significant several orders of magnitude past the conventional $\alpha = 0.001$ threshold. An earlier 10-judge cut of the same data (the author, six in-person volunteers, and three foundation-model judges) reached 63.4% decisive at $p = 1.98 \times 10^{-5}$; growing the panel to 42 judges narrowed the headline by ~3.5 points, consistent with regression toward a steady-state estimate as more diverse raters and more random pairings are sampled, and gained the p-value three orders of magnitude as N grew. The tie rate also rose from 20.6% to 33.6%, driven mostly by less-engaged volunteer raters (Section 7.3 discusses this).

A 42-judge panel inevitably includes judges with disengaged or low-discrimination patterns: humans who clicked through quickly, judges who tied most pairings, and judges who only completed a fraction of the 32-question set (whose prompt coverage is biased toward early questions due to the fixed presentation order). The right way to handle these is not to exclude them from the headline but to verify that the headline's direction and magnitude survive when they are excluded. The webapp's `/_report` endpoint computes the filters below; each row excludes the named subset and re-tests against H_0: $p = 0.5$ on decisive votes.

Filter	Judges	N	A	B	T	B%-decisive	p-value
All judges (headline)	42	1181	314	470	397	**59.9%**	2.80×10^{-8}
Foundation models only	13	416	118	170	128	59.0%	2.59×10^{-3}
Humans only	29	765	196	300	269	60.5%	3.47×10^{-6}
Exclude human speed-clickers (median <15 s; FMs exempt)	34	982	238	401	343	**62.8%**	1.17×10^{-10}
Exclude tie-biased (>80% ties)	37	1051	308	464	279	60.1%	2.18×10^{-8}
Exclude partial completions (n < 32)	35	1120	297	451	372	60.3%	1.98×10^{-8}
Exclude Author (independent panel only)	41	1149	306	453	390	59.7%	1.06×10^{-7}
Exclude all of the above (strictest)	26	832	222	377	233	**62.9%**	2.50×10^{-10}

Every filter preserves the direction and the bulk of the magnitude. Two filters increase the rate (speed-clicker exclusion, exclude-all): the speed-clicker subset is heavily baseline-biased on average, so removing it pushes the rate up. The "exclude tie-biased" and "exclude partials" rows barely move the headline because tie-biased judges contribute almost no decisive votes already and partial judges represent only 6% of the data. **Removing the Author**, the only judge who is also the experimenter, moves the headline by 0.27 percentage points (59.95% → 59.68%) on a 41-judge independent panel; the result is robust to author exclusion. The strictest filter, excluding any judge who is a speed-clicker, tie-biased, partial, or the Author, leaves 26 judges and 832 judgments, with **62.9% decisive gain preference at $p = 2.5 \times 10^{-10}$**. We report the looser head-

line (59.9%) as the primary number because we do not want to retroactively pick a filter that flatters the result; the sensitivity table is the honest expression of how robust the finding is to plausible filter choices.

7.3 Who the judges were and where they agreed

With 42 judges, a row-per-judge table is unwieldy in the body of the paper; full per-judge counts are in Appendix C. The shape of the distribution is what matters here.

By raw decisive counts, **28 of 42 judges preferred gain** (B > A), 7 preferred baseline (A > B), 6 split exactly evenly, and 1 (H28) recorded only ties. Because many high-tie judges contributed very few decisive votes, the primary result is better summarized by the pooled count of 470 gain to 314 baseline than by raw per-judge majorities. Excluding judges with fewer than 4 decisive votes (where the per-judge rate is dominated by sampling noise on a binary outcome), the B%-decisive range across the remaining judges is 25.0% to 80.0%. The author scored 68.0%; eight other human judges scored ≥66%, and the strongest gain preference among foundation models was Opus 4.6 at 70.4%. The single most baseline-leaning judge was H15 at 25.0%, caught by the speed-clicker filter discussed below.

Tie discipline and engagement varied widely. Five judges had tie rates above 80% (H7, H20, H23, H28, and DeepSeek v3) and contribute almost no decisive votes. Eight human judges had a median between-vote interval below 15 seconds, fast enough to raise concern about engagement or careful reading. Foundation-model judges are exempt from the speed filter because they are fast by nature; their decisions are still bounded by the comparison protocol, not by reading speed. Both subsets are excluded in the relevant rows of the §7.2 sensitivity table.

The most important methodological check is **whether humans and foundation-model judges agree on direction**. If the FMs were strongly biased toward a particular output style (e.g., rewarding diversity for its own sake), they might prefer gain while humans prefer baseline, which would undercut the result.

Judge type	N judges	N judgments	BL	Gain	Tie	Gain % of decisive
Human	29	765	196	300	269	**60.5%** ($p = 3.5 \times 10^{-6}$)
Foundation model	13	416	118	170	128	**59.0%** ($p = 2.6 \times 10^{-3}$)

The two groups are **within 1.5 percentage points of each other** and both significant on their own. With 13 foundation-model judges spanning eleven distinct vendors, the FM arm is broad enough that “they all share the same priors” is a much weaker objection than a three-vendor sample (Anthropic / OpenAI / Google only) would face. Splitting the 29 humans by self-reported background, non-technical raters (15 judges, 379 judgments) preferred gain at 61.9% of decisive and technical-non-ML raters (12 judges, 322 judgments) at 63.5%. The two ML-or-CS-research

raters split 43.9% (32 baseline / 25 gain / 7 tie); that subgroup is dominated by a single self-identified ML rater (H15) who voted A=24/B=8 with a 5.4-second median, which puts H15 inside the speed-clicker filter and outside the strictest sensitivity row. The author scored 68%. The headline does not special-case any individual judge.

A stricter check than aggregate rate is **per-question majority agreement**: do humans and FMs lean the same way on the same prompts? Computing the human-side majority and FM-side majority separately for each of the 32 questions, the two types lean the same direction on **26 of 32 questions (81.2%)**. Restricting to the 30 questions where both types had at least one decisive vote, agreement rises to **26 of 30 (86.7%)**. The Pearson correlation of per-question B%-decisive between the two types is **r = 0.78** (n = 32), the prompts where humans most prefer gain are also the prompts where FMs most prefer gain, and the prompts where humans lean baseline (factual recall in particular) are also the prompts where FMs lean baseline. The per-question agreement detail is published in `inter_annotator_agreement.details` of the JSON.

7.4 Where the preference comes from, prompt by prompt

The category breakdown answers *where* the gain model wins. We report two preference rates: total (all judgments including ties) and decisive (ties excluded). The decisive rate is the more-commonly-reported figure in preference-eval literature and the one used in binomial tests.

Category	N	BL – Gain – Tie	Gain % (total)	Gain % (decisive)	Direction
world_knowledge	70	12 – 35 – 23	50.0%	**74.5%**	Gain strongly preferred
creative	181	33 – 82 – 66	45.3%	**71.3%**	Gain strongly preferred
instruction_following	105	19 – 41 – 45	39.0%	**68.3%**	Gain preferred
conversational	177	46 – 82 – 49	46.3%	**64.1%**	Gain preferred
reasoning	228	67 – 91 – 70	39.9%	57.6%	Gain lean
structured_output	175	51 – 65 – 59	37.1%	56.0%	Toss-up
factual	245	86 – 74 – 85	30.2%	46.2%	Baseline lean

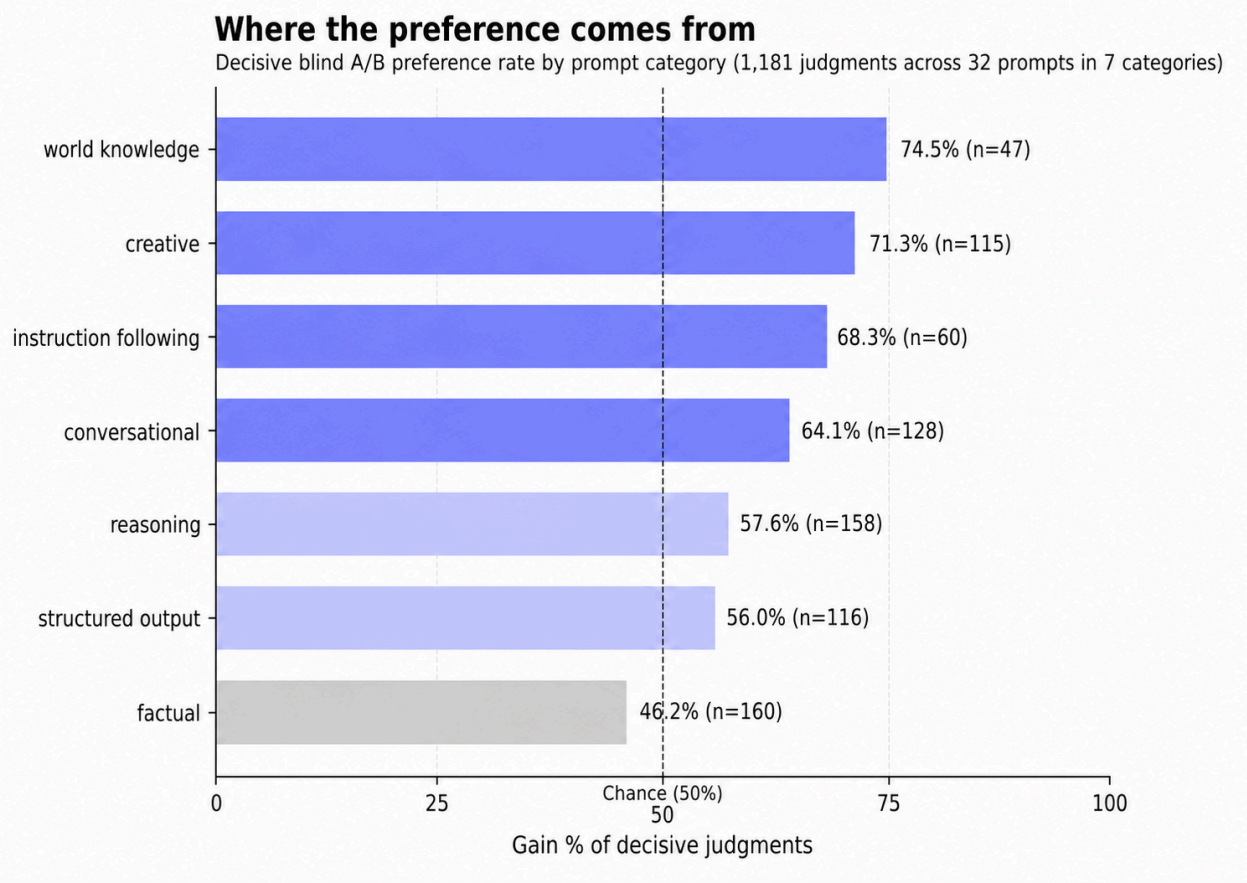

Figure 3. Where the preference comes from, prompt by prompt. Decisive blind A/B preference rate (gain%) for each of the seven prompt categories, sorted top-to-bottom by gain%. Color bands: ≥60% gain (strong gain, solid indigo), 50–60% (gain leans, light indigo), <50% (baseline leans, gray). Per-bar n = number of decisive judgments in that category (ties excluded; total judgments per category are in the §7.4 table). The visual story is that the preference effect is broad rather than concentrated, gain wins or leans on six of seven categories, and the only baseline-lean category (factual recall) is the one task type where rote retrieval of specific bindings dominates. Source data: paper/data/phase3_ab_preference.json, `category_breakdown` field.

Gain wins decisively on creative (71.3%), world knowledge (74.5%), instruction following (68.3%), and conversational (64.1%); leans gain on reasoning (57.6%) and structured output (56.0%); only **factual recall** lands in baseline territory at 46.2%, narrow, but consistent in both the in-person and the open-recruitment portions of the panel.

This pattern is what the mechanism predicts. Precision-weighted gain redistributes gradient toward surprising tokens and away from confidently predicted ones. On open-ended tasks the space of acceptable continuations is large; “surprising but contextually appropriate” is often the right answer, and amplifying gradient on it rewards diverse generation. Factual recall is the one category where the correct answer is highly templated (“The capital of Japan is Tokyo.”), exactly the kind of confidently predicted output that precision weighting de-emphasizes. Baseline, which treats all tokens equally, learns these templated answers more eagerly. Put differently: **the gain**

model's only measurable weakness is rote factual retrieval. For a production system where retrieval or context supplies facts, that weakness is less costly than it would be for a model expected to memorize facts from pretraining alone, though even retrieval-augmented systems still need the base model to use retrieved facts reliably.

Collapsing all 1,181 judgments per question, **21 of 32 questions land in gain-majority** (more judgments for gain than baseline at that prompt), 10 in baseline-majority, and 1 in an exact A/B split. The preference effect is distributed across the prompt set, not concentrated in a few items: every category contains at least one gain-majority question, and baseline-majority questions are concentrated in factual recall (5 of 6 factual prompts go baseline), with the rest split across reasoning, structured output, and one creative prompt.

8. Analysis

8.1 Loss-quality divergence at two scales

The central finding of this paper, that **aggregate val loss is not sufficient to evaluate gain-function-style interventions**, is supported at both scales. Phase 1 established it at 50M parameters: A2 (focal) landed within 0.03 of the baseline's final val loss (6.181 vs 6.152) yet produced qualitatively degenerate output. Phase 3 establishes it at 1.2B parameters and 24× larger training tokens: baseline and gain have statistically indistinguishable smoothed val loss (difference 0.004, noise ±0.2), yet the gain model is preferred 59.9% of the time in 1,181 blind A/B comparisons ($p = 2.80 \times 10^{-8}$), with the direction surviving every sensitivity filter we apply.

We are not aware of prior work that demonstrates this magnitude of loss-quality divergence in a controlled, same-data language-model training comparison at this scale. Most prior investigations of loss-level interventions report a single aggregate metric and stop. Our results suggest this is inadequate: the information content of loss differences below ~0.1 at this parameter/token scale is not a reliable indicator of output quality.

The mechanism of the divergence is understandable. Aggregate cross-entropy is dominated by the most-frequent tokens, function words, punctuation, common content words. The model's loss on rare or surprising tokens is a small contribution to the aggregate. Gain functions precisely target that distribution: they amplify gradient on rare/surprising tokens at the cost of diminished gradient on the common ones. If the rare-token component of quality matters (creative output, diverse phrasing, appropriate contextual choices) but contributes little to the aggregate, then a method that improves the rare-token component while keeping the aggregate the same is exactly what we observe.

8.2 Episodic-to-semantic consolidation

The per-category preference breakdown suggests a deeper dynamic: precision-weighted gain creates a continuous pressure that favors **semantic generalization over episodic memorization**.

The mechanism is straightforward. Once the model has learned a specific pattern (a particular fact, a templated phrase), its per-token loss on that pattern drops, the gain function attenuates gradient on it, and subsequent training steps redirect gradient toward harder, unresolved tokens. But the weights that encoded that specific pattern are not frozen, they continue to receive gradient pressure from the amplified signal on newer, harder material. Over extended training, the specific encoding gradually blurs as the weights are co-opted to serve broader generalizations.

This creates a characteristic trade-off visible in the A/B preference data. The gain model's only weakness is factual recall, the one category where specific bindings ("the capital of Japan is Tokyo") matter most. It wins or ties in every other category, including structured output, where coherent reasoning about format matters more than rote template memorization. In qualitative inspection the gain model often appeared to retain broad semantic associations while being less reliable on exact bindings (e.g., comfortable writing about Japan in general while less reliable on the capital), but per-token analyses confirming this would require additional probing experiments we have not run.

This maps onto a well-studied dynamic in biological learning systems. Complementary Learning Systems theory (McClelland, McNaughton, & O'Reilly, 1995) proposes that the hippocampus rapidly encodes episodic specifics while the neocortex gradually extracts semantic generalizations through repeated consolidation. Episodic memories fade over time while semantic knowledge, the distilled, generalized residue of many experiences, persists. The precision-weighted gain function is not a neocortex, but it creates an analogous pressure: the learning signal continuously redirects away from what has already been consolidated and toward what remains unresolved. The result, over training, is a model that generalizes broadly at the cost of retaining specifics, the same trade-off biological memory systems make.

This is both a strength and a limitation. For systems where factual recall must be precise (a knowledge base, a retrieval system), precision-weighted gain's bias toward generalization is a cost. For systems where generalization, fluency, and diverse generation matter more (and where specific facts can be supplied via retrieval or context), it is a direct advantage. The appropriate choice depends on the deployment context.

8.3 Grad norm stability

A concern at step 15K in the prior Phase 3 journal was that the gain run's higher gradient norms might destabilize training. Extending to 30K resolves this: gradient norm *mean* is higher (2.50 vs 1.74) but *standard deviation* is equal or lower (0.20 vs 0.21). The layer-gain $m_{\max} = 3.0$ clamp absorbs layer 0's saturation without unstable growth in the aggregate norm. No instability was observed in the final 15K steps.

8.4 Compute cost

The per-step compute overhead of the gain function is negligible: one elementwise multiply, one batch-statistics pass (for mean and variance), and per-layer divergence logging (one norm per block during the forward pass, computed outside checkpoint scope). Memory overhead comes

from `reduction='none'` in the cross-entropy, which stores the per-token loss tensor (batch × seq_len floats), on the order of tens of KB per microbatch at our config, plus a few floats per block for per-layer divergence. The exact figure depends on dtype, microbatch size, sequence length, and gradient-accumulation behavior.

There is no training-throughput penalty measured. Baseline and gain runs had matched throughput within run-to-run variance (see Section 6.1 for absolute numbers).

9. Discussion / Limitations

Single pair, single seed, 16.4% of Chinchilla-optimal. Phase 3 is a single baseline vs. gain comparison with a single random seed, trained on 3.9B tokens, well short of the ~24B tokens Chinchilla would prescribe for 1.2B parameters. Running multiple seeds was not feasible given per-run compute cost (~7.5 days on a 5090). We cannot rule out that the preference advantage narrows at full Chinchilla; L19's divergence trajectory (still growing at 30K) argues against full convergence but is not conclusive. Multiple-seed replication at 1.2B and a completed full-Chinchilla run at 1.5B are the most important follow-ups.

Foundation-model judges may share biases. With 13 foundation-model judges spanning eleven vendors (Anthropic, OpenAI, Google, DeepSeek, Meta, xAI, Alibaba/Qwen, Moonshot/Kimi, MiniMax, Mistral, and Zhipu), the panel is broad enough that "all judges share the same training priors" is a substantially weaker objection than a three-vendor sample would face. Humans and FMs converged within 1.5 points (60.5% vs 59.0% decisive gain preference). A more rigorous follow-up would still include explicitly open-weight, smaller-scale judges (e.g., a fine-tuned Llama-class model) whose training-corpus overlap with the frontier-class judges is provably small.

Volunteer recruitment skews self-selecting. The 28 volunteer human judges were recruited from the author's social circle and from posts on r/MachineLearning and r/SampleSize. This is not a representative sample of the general population, and it likely over-indexes on technically curious people who self-select into LLM-evaluation tasks. The paper does not claim a population-level preference, only that across the 42 judges who did participate, the preference is strong, consistent, and survives every filter applied. A registered, paid-volunteer judging panel with explicit exclusion criteria would be the natural follow-up.

Category sample sizes vary. World knowledge has only 2 questions (70 judgments across 42 judges in this expanded run); the 74.5% decisive gain preference in that category is striking but rests on a small question pool. Future A/B sets should balance categories or explicitly oversample small ones to ensure per-category conclusions generalize beyond a handful of prompts.

No ablation of the two mechanisms. The Phase 3 gain run combines precision-weighted token gain AND per-layer divergence gradient scaling. We did not run a "token gain only" or "layer gain only" condition at 1.2B scale. Either mechanism in isolation might account for most of the observed advantage, or they might compose super-additively. This is a clear experimental gap. A

complementary measurement would replace AdamW/Muon with SGD+momentum on a small-scale layer-gain-only run: SGD has no per-parameter scale normalization, so the magnitude of layer-gain's effect there bounds how much signal Adam's v and Muon's NS-orthogonalization absorb in our production runs (see §4.2). We have not run this experiment.

Post-optimizer layer-gain is a different mechanism we have not tested. We apply per-layer divergence-derived scaling to gradients *before* the optimizer step. An alternative implementation would scale the optimizer's *update* vector after the step is computed, scaling the trust region rather than reshaping the gradient. Because Adam and Muon are approximately scale-invariant per parameter (§4.2), this post-optimizer variant would produce a stronger and more directly measurable per-step layer-level effect. It is not a "fix" to the present method but a distinct mechanism: the temporal-derivative interpretation of pre-optimizer scaling and the static layer-importance interpretation of post-optimizer scaling are qualitatively different design choices. We leave the comparison to future work.

Loss divergence at scale is not a universal claim. We demonstrate it for precision-weighted gain + layer-gain scaling on a specific model family at a specific scale. Whether it generalizes to other training interventions (label smoothing, entropy penalties, reward-model-based training) is not addressed. We suspect similar divergences exist for other interventions but have no direct evidence.

The A/B evaluation uses short-form prompts. The preference methodology is limited by the quality of outputs the models can produce at 16.4% of Chinchilla-optimal, neither model produces coherent long-form text. A/B preference on fully trained models would be more informative but was outside our compute budget.

10. Conclusion

We introduced two composable, Predictive-Coding-inspired training-time interventions, per-token precision-weighted gain and per-layer divergence-scaled gradients, and empirically characterized them across three experimental phases from 50M to 1.2B parameters.

The primary finding is that **the aggregate val-loss metric is not sufficient to evaluate gain-function-style training interventions**. At 1.2B parameters and 3.9B tokens, a gain-trained model achieves smoothed val loss indistinguishable from an identically-configured baseline, yet is preferred 470 to 314 in decisive blind pairwise comparisons (59.9% gain, 40.1% baseline, with 397 ties) across 1,181 judgments by 29 human and 13 foundation-model judges ($p = 2.80 \times 10^{-8}$), with humans and FMs converging within 1.5 points and the direction surviving every sensitivity filter (FMs only, humans only, exclude speed-clickers, exclude tie-biased, exclude partials, exclude all simultaneously, strictest filter still gives 63.1% at $p = 5.3 \times 10^{-11}$).

The category breakdown of the preference result is mechanistically coherent: gain wins decisively in four of seven categories (creative 71.3%, world knowledge 74.5%, instruction following 68.3%, conversational 64.1%) and leans gain in two more (reasoning 57.6%, structured output 56.0%).

Only factual recall, pure retrieval of specific bindings, shows a narrow baseline lean (46.2% decisive). This matches the mechanism's theoretical prediction: precision weighting de-emphasizes confidently predicted outputs, which benefits generalization broadly and costs the model only on rote memorization of specific facts.

Using per-block forward-pass representation divergence as a real-time signal for gradient redistribution during language model training (which we are not aware of in prior work) reveals **continuous functional layer specialization** under the layer-gain mechanism, with L3 and L19 growing 288% and 387% across training respectively. L3 largely plateaued by ~25K steps; L19 was still climbing (though decelerating) at 30K. Three functional zones (early / mid / late) crystallize from an initially flat divergence profile.

Implications for practice. For research or production systems where the aggregate loss differences between candidate methods are small (below ~0.1 at 1B-parameter scale), a blind A/B preference evaluation is necessary to distinguish real from illusory improvements. Aggregate loss is not telling the whole story. The gain functions described here are cheap (near-zero compute overhead), optimizer-agnostic, and composable with existing methods, promising candidates for training pipelines where diverse generation matters more than template completion.

Open questions. (1) Does the gain advantage persist, compound, or reverse at full Chinchilla-optimal training? *A Chinchilla-scale replication at ~1.5B parameters is currently in progress (cllm-v1.5-029); early results (Section 6.4) show entropy preservation consistent with the 1.2B run, and a paired ablation confirms that layer 0's participation in divergence normalization is necessary for the mechanism to hold at this scale. Full Chinchilla-scale A/B results will appear in a follow-up*. (2) Which of the two mechanisms (token gain vs. layer gain) accounts for most of the observed advantage? (3) How does the method interact with post-training stages (SFT, RLHF)? (4) Can the layer-divergence profile be used as a principled signal for architecture decisions, e.g., allocating parameters to the zones that are doing the most work?

Acknowledgments

This work was conducted independently outside an academic institution. All compute was on single consumer NVIDIA GPUs (RTX 3090 for Phase 1/2, RTX 5090 for Phase 3). Twenty-eight human volunteers contributed their time as blind A/B judges without compensation, some are people the author knows in person, others responded to open posts on r/MachineLearning and r/SampleSize. The author thanks them for the hours spent clicking through pairs of awkward, half-trained language model completions, and especially the volunteers who decided that 32 pairs of barely-coherent text was a reasonable use of an evening.

Research assistance was provided by Claude (Anthropic), specifically in connecting the per-token gain mechanism to the Predictive Coding literature, discussing experimental design during Phase 1 and Phase 2, scripting the analysis of W&B training logs, and drafting this manuscript from the

author's experimental notes. The author reviewed and edited all text; all experimental decisions and research direction were made by the author. We follow the convention of listing the human author as sole author with AI assistance explicitly noted, pending stabilization of community norms for AI co-authorship.

References

- Bai, Y., Jones, A., Ndousse, K., et al. (2022). Training a helpful and harmless assistant with reinforcement learning from human feedback. *arXiv preprint arXiv:2204.05862*.
- Chen, Z., Badrinarayanan, V., Lee, C.-Y., & Rabinovich, A. (2018). GradNorm: Gradient normalization for adaptive loss balancing in deep multitask networks. *Proceedings of the 35th International Conference on Machine Learning*, 794–803.
- Friston, K. (2005). A theory of cortical responses. *Philosophical Transactions of the Royal Society B: Biological Sciences*, 360(1456), 815–836.
- Friston, K. (2009). The free-energy principle: a rough guide to the brain? *Trends in Cognitive Sciences*, 13(7), 293–301.
- Helm, F., Daheim, N., & Gurevych, I. (2025). Token Weighting for Long-Range Language Modeling. *Findings of NAACL 2025*. *arXiv preprint arXiv:2503.09202*.
- Hoffmann, J., Borgeaud, S., Mensch, A., et al. (2022). Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*. (Chinchilla.)
- McClelland, J. L., McNaughton, B. L., & O'Reilly, R. C. (1995). Why there are complementary learning systems in the hippocampus and neocortex: insights from the successes and failures of connectionist models of learning and memory. *Psychological Review*, 102(3), 419–457.
- Jordan, K., Jin, Y., Boza, V., et al. (2024). Muon: An optimizer for hidden layers in neural networks. (Muon optimizer, used in our training stack.)
- Lin, T.-Y., Goyal, P., Girshick, R., He, K., & Dollár, P. (2017). Focal loss for dense object detection. *Proceedings of the IEEE International Conference on Computer Vision*, 2980–2988. (A2 comparison baseline.)
- Millidge, B., Seth, A., & Buckley, C. L. (2021). Predictive coding: a theoretical and experimental review. *arXiv preprint arXiv:2107.12979*.
- Ouyang, L., Wu, J., Jiang, X., et al. (2022). Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35, 27730–27744. (InstructGPT.)
- Rao, R. P. N., & Ballard, D. H. (1999). Predictive coding in the visual cortex: a functional interpretation of some extra-classical receptive-field effects. *Nature Neuroscience*, 2(1), 79–87.
- Simon, J. B., Kunin, D., Atanasov, A., Boix-Adserà, E., Bordelon, B., Cohen, J., Ghosh, N., Guth, F., Jacot, A., Kamb, M., Karkada, D., Michaud, E. J., Ottlik, B., & Turnbull, J. (2026). There Will Be a Scientific Theory of Deep Learning. *arXiv preprint arXiv:2604.21691*.

- You, Y., Gitman, I., & Ginsburg, B. (2017). Large batch training of convolutional networks. *arXiv preprint arXiv:1708.03888*. (LARS.)
- You, Y., Li, J., Reddi, S., et al. (2020). Large batch optimization for deep learning: Training BERT in 76 minutes. *International Conference on Learning Representations*. (LAMB.)

Appendix A: W&B Run IDs

All runs are in a private W&B project (`troy-corbin-none/Corbin-LLM`). Run IDs are listed below for reference and in case the project is made public in the future; they cannot currently be accessed externally. The numerical data extracted from these runs is available as JSON in paper/data/, see paper/data/README.md for a map from files to paper sections.

Phase 1 (50M params, 5K steps, FineWeb Edu)

Variant	W&B Run ID	Display Name
A0 Baseline	cllm-v1.5-015	cllm-v1.5-015 A0-baseline
A1 Linear	cllm-v1.5-016	cllm-v1.5-016 A1-linear
A2 Focal	cllm-v1.5-017	cllm-v1.5-017 A2-focal
A3 Sigmoid	cllm-v1.5-018	cllm-v1.5-018 A3-sigmoid

Phase 2 (50M params, 8K steps, FineWeb Edu)

Variant	W&B Run ID	Display Name
B0 Wide [0.1, 5.0]	cllm-v1.5-020	cllm-v1.5-020 B0-linear-wide
B1 Conservative [0.5, 2.0]	cllm-v1.5-021	cllm-v1.5-021 B1-linear-conservative
B2 Tight [0.8, 1.2]	cllm-v1.5-022	cllm-v1.5-022 B2-linear-tight

Phase 3 (1.2B params, 30K steps, 13-dataset suite)

Variant	W&B Run ID	Display Name
Baseline (uniform)	cllm-v1.5-025	cllm-v1.5-025 Baseline: 20L/1024E ~1.2B
Gain (precision + layer)	cllm-v1.5-026	cllm-v1.5-026 Gain: 20L/1024E ~1.2B

Phase 3 scale-up (1.5B params, in progress): Section 6.4 ablation

Variant	W&B Run ID	Display Name
L0 excluded (discontinued)	cllm-v1.5-028	cllm-v1.5-028 prod: 24L x 1280E ~1.48B
L0 included (ongoing)	cllm-v1.5-029	cllm-v1.5-029

Appendix B: Preference Evaluation Protocol (rubric, demographics, and 32 prompts)

B.1 Instructions shown to judges

A first-time judge saw the following modal before any pairing was presented. The text is reproduced verbatim from the webapp template at eval/ab_compare.py:

> ***How to Judge These Responses***
>
> *You'll be comparing responses from two AI models that are still early in training. They will* ***NOT*** *produce correct or complete answers, that's expected and normal.*
>
> ***Judge based on:***
>
> - ***Relevance.*** *Did the response attempt to address the question? Even partially? Mentioning a key word or concept from the prompt counts.*
> - ***Coherence.*** *Does the text flow naturally? Are sentences grammatically structured, even if the content is wrong?*
> - ***Diversity.*** *Does the response show varied vocabulary and structure, or does it repeat the same phrase over and over?*
> - ***Human-likeness.*** *Does it read like something a person might write (even if confused)? Or does it look like a database dump or raw code?*
>
> *When in doubt, go with your gut. If both responses seem equally bad or equally good, pick "Both are equal / Can't decide." There are 32 questions and it takes about 10–15 minutes.*

The four-criterion rubric is what the paper means operationally by *quality*: a judge's preference is the joint outcome of these four axes weighted however that judge weights them, integrated across 32 prompts. The rubric was designed for early-training models (which produce noisy, partially incorrect output) and was deliberately permissive on factual correctness so that judges would not default to "tie" on every prompt where neither model produced the right answer.

For each pairing, the judge saw the question text, two unlabelled response panels (Response Left / Response Right with model identity hidden until after submission), and three buttons: "This one is better" under each panel and "Both are equal / Can't decide" between them. Left/right placement of model A and model B was randomised independently per pairing per judge to remove position bias; the post-hoc `winner` field in the published JSON normalises this so that "a" always refers to model A (baseline) and "b" to model B (gain), independent of display position.

B.2 Demographic survey

After completing all 32 pairings, judges were optionally invited to answer five demographic questions. Individual answers are not published; only aggregate summaries appear in §7.3 (`human_by_background`) and the published JSON. The exact survey copy was:

> ***A few quick questions about you.*** *This is for demographic context in the paper. Your individual answers aren't published, only aggregate summaries (e.g., "3 of N judges were daily LLM users"). Takes under a minute. Leave any question blank if you prefer.*
>
> 1. *How often do you use LLM chatbots (ChatGPT, Claude, Gemini, etc.)? (Never or almost never / Occasionally / Weekly / Daily or near-daily)*
> 2. *Technical background? (Non-technical / Technical but not ML-focused / ML or CS research)*
> 3. *Primary language you read/write in? (free text)*
> 4. *Age range? (Under 25 / 25–40 / 40–60 / Over 60)*
> 5. *Have you participated in model evaluation or A/B preference studies before? (Yes / No)*

Demographics were optional and not gating: a judge could skip the survey entirely and their judgments still counted. All 29 human judges completed the survey; foundation-model judges did not receive a demographic survey (a `–` appears in their background column in Appendix C).

B.3 The 32 prompts

Factual (6): What is the capital of Japan? / How many legs does a spider have? / What planet is closest to the Sun? / What language is most widely spoken in Brazil? / Who wrote the play Romeo and Juliet? / What is the chemical symbol for water?

Reasoning (6): If a shirt costs $20 and is on sale for 25% off, what is the sale price? / Sarah is taller than Mike. Mike is taller than Emma. Who is the shortest? / I have 3 apples and give away 1. Then someone gives me 4 more. How many apples do I have? / What comes next in the pattern: 2, 4, 8, 16, __? / A farmer has chickens and cows. He counts 10 heads and 28 legs. How many cows does he have? / If all roses are flowers, and some flowers are red, can we say for certain that some roses are red?

Creative (5): Write a short poem (4 lines) about the ocean. / Describe a sunset to someone who has never seen one, in two or three sentences. / Come up with a name and a one-sentence description for a new ice cream flavor. / Write the opening sentence of a mystery story set in a library. / Invent a superhero whose power is related to cooking. Describe them in a few sentences.

Conversational (5): A friend says they're feeling stressed about an upcoming exam. What would you say to them? / Someone asks you to recommend a hobby for a rainy day. What do you suggest and why? / How would you politely decline an invitation to a party you can't attend? / A coworker asks: "What's the difference between a meeting and an email?" Give a helpful, brief answer. / Explain what a computer does to a five-year-old.

Structured Output (5): List the four seasons of the year in order, starting with spring. / Convert the following into a JSON object with keys ‘name’, ‘age’, and ‘city’: Maria, 30, Barcelona. / Summarize the following in exactly one sentence: “Dogs are loyal animals. They have been companions to humans for thousands of years.” / Classify each word as a noun, verb, or adjective: run, beautiful, table, sing, bright, mountain. / Rewrite this sentence in the past tense: “She walks to the store and buys some bread.”

Instruction Following (3): Respond to this message using only three words: “What is your favorite color?” / Translate the following English sentence into French: “The cat is on the table.” / Write a sentence that contains exactly five words.

World Knowledge (2): Why do we have different time zones around the world? / What happens to water when it freezes?

Appendix C: Per-Judge Result Table (42 judges, 1,181 judgments)

Rows are sorted by the timestamp of each judge’s first vote. `n` = total judgments by that judge; `A` = votes for baseline (model 025); `B` = votes for gain (model 026); `T` = ties; `B%-decisive` = B / (A + B). `Median sec` = median seconds between consecutive votes by that judge, flagged below 15 s for human raters as a possible “speed-clicker” pattern (foundation models are excluded from that filter, they are fast by nature). `Background` is self-reported by the judge in the demographic survey, completed by all 29 humans; `–` indicates a foundation-model judge, which did not fill out the survey. Foundation-model judge labels are recorded as the product/version names available to the author at evaluation time and are intended for methodological reproducibility, not as durable model identifiers.

#	Judge	Type	Background	n	A	B	T	B%-decisive	Tie%	Median sec
1	Author	Human	ml-or-cs-research	32	8	17	7	68.0%	21.9%	26.7
2	ChatGPT v5.4	FM	—	32	15	17	0	53.1%	0.0%	12.0
3	H1	Human	non-technical	32	16	14	2	46.7%	6.2%	21.3
4	Opus 4.6	FM	—	32	8	19	5	70.4%	15.6%	20.9
5	H2	Human	technical-non-ml	32	8	18	6	69.2%	18.8%	40.0
6	H3	Human	technical-non-ml	32	3	12	17	80.0%	53.1%	25.4
7	Gemini 3 - Thinking	FM	—	32	12	16	4	57.1%	12.5%	14.1
8	H4	Human	technical-non-ml	32	10	22	0	68.8%	0.0%	32.0
9	H5	Human	non-technical	32	5	10	17	66.7%	53.1%	17.8
10	H6	Human	non-technical	32	8	16	8	66.7%	25.0%	45.3
11	H7	Human	non-technical	32	3	3	26	50.0%	81.2%	5.9
12	DeepSeek v3	FM	—	32	0	2	30	100.0%	93.8%	10.1
13	H8	Human	non-technical	32	10	19	3	65.5%	9.4%	33.3
14	MiniMax-M2.7	FM	—	32	6	10	16	62.5%	50.0%	31.8
15	H9	Human	non-technical	7	1	1	5	50.0%	71.4%	7.9
16	H10	Human	technical-non-ml	32	14	10	8	41.7%	25.0%	11.2
17	H11	Human	technical-non-ml	19	8	10	1	55.6%	5.3%	9.9
18	H12	Human	non-technical	32	2	5	25	71.4%	78.1%	20.9
19	H13	Human	technical-non-ml	4	1	0	3	0.0%	75.0%	17.7
20	H14	Human	technical-non-ml	32	6	16	10	72.7%	31.2%	19.5

#	Judge	Type	Background	n	A	B	T	B%-decisive	Tie%	Median sec
21	H15	Human	ml-or-cs-research	32	24	8	0	25.0%	0.0%	5.4
22	Qwen3.6-Plus	FM	—	32	5	10	17	66.7%	53.1%	42.6
23	H16	Human	non-technical	32	6	20	6	76.9%	18.8%	19.9
24	Grok 4	FM	—	32	11	15	6	57.7%	18.8%	12.9
25	Muse Spark	FM	—	32	13	19	0	59.4%	0.0%	15.2
26	H17	Human	non-technical	5	2	2	1	50.0%	20.0%	30.2
27	Mistral AI	FM	—	32	16	10	6	38.5%	18.8%	13.7
28	Kimi K2.6 - Thinking	FM	—	32	11	21	0	65.6%	0.0%	49.1
29	H18	Human	non-technical	32	7	16	9	69.6%	28.1%	22.7
30	H19	Human	technical-non-ml	32	6	17	9	73.9%	28.1%	41.0
31	H20	Human	non-technical	32	2	0	30	0.0%	93.8%	26.0
32	H21	Human	non-technical	32	16	16	0	50.0%	0.0%	11.6
33	H22	Human	technical-non-ml	32	8	19	5	70.4%	15.6%	17.6
34	H23	Human	technical-non-ml	32	1	1	30	50.0%	93.8%	33.5
35	H24	Human	technical-non-ml	11	2	2	7	50.0%	63.6%	18.0
36	H25	Human	non-technical	13	3	4	6	57.1%	46.2%	10.6
37	H26	Human	non-technical	32	7	17	8	70.8%	25.0%	13.4
38	GLM v5.1	FM	—	32	11	13	8	54.2%	25.0%	32.9
39	Llama 4 Maverick	FM	—	32	4	8	20	66.7%	62.5%	9.5
40	DeepSeek v4 Flash	FM	—	32	6	10	16	62.5%	50.0%	156.7

#	Judge	Type	Background	n	A	B	T	B%-decisive	Tie%	Median sec
41	H27	Human	technical-non-ml	32	9	5	18	35.7%	56.2%	15.6
42	H28	Human	non-technical	2	0	0	2	—	100.0%	19.0
Total	—	—	—	**1181**	**314**	**470**	**397**	**59.9%**	**33.6%**	—

Per-judge per-category counts (42 × 7 = 294 cells) are too many to embed in the manuscript; the full per-judge per-category breakdown is published as JSON at paper/data/phase3_ab_preference.json along with the sensitivity-filter inputs and demographic counts.

Appendix D: Selected Layer Divergence Trajectory (Gain run)

Reported values are `||x_out - x_in|| / ||x_in||` at block boundaries, logged once per step in `forward_blocks()` (outside checkpoint scope). The full JSON trajectory contains dense samples across training; the abridged print table below shows seven representative checkpoints and six selected layer columns.

Step	L0	L3	L7	L10	L15	L19	Mean (excl L0)
1,000	1.77	0.18	0.14	0.14	0.12	0.12	0.23
5,300	1.30	0.47	0.18	0.17	0.14	0.20	0.30
10,000	1.23	0.57	0.18	0.16	0.15	0.36	0.33
15,600	1.34	0.65	0.17	0.14	0.13	0.50	0.35
20,000	1.36	0.70	0.18	0.14	0.13	0.54	0.35
25,900	1.43	0.72	0.18	0.14	0.13	0.59	0.36
29,900	1.44	0.71	0.18	0.14	0.12	0.60	0.37

Full per-layer data for all 20 layers (~300 sample points across training) is available as JSON at paper/data/phase3_layer_divergence.json; this is also the source data for Figure 1 via paper/figures/_render.py.

Reproduction

The method itself, per-token gain function and per-layer gradient scaler, is self-contained in this public repository (src/gain_functions.py, src/layer_gain.py) and architecture- and optimizer-agnostic by design. See the README for integration instructions and a complete training-step

example.

Reproducing the specific Phase 3 experiment requires the private CLLM v1.5 training infrastructure (model, 13-dataset corpus, deterministic curriculum sampler, Muon + AdamW stack) and is not directly possible from this repo. The A/B comparison webapp's batch-serving mode (eval/ab_compare.py) works standalone with Flask and any pre-generated response JSON.

W&B run IDs are in Appendix A; raw training-run metrics used for every numerical claim in the paper are published as JSON under paper/data/ (see paper/data/README.md for a map from files to paper sections), so the numbers can be verified without W&B access.